

SIMATS ENGINEERING ASSESSMENT TOOLS

COURSE NAME: Theory of Computation

COURSE CODE: CSA13

COURSE OUTCOME COVERED

CO4: *Construct Context-Free Grammars, generate derivations and parse trees to demonstrate the syntactic structure of strings. (BL : 3)*

Assessment Tool Weightage: 20%

QUESTIONS: 5

Total Marks:25

Duration : 1 Hour

Industry Problem

Code of Conduct:

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

SIMATS ENGINEERING ASSESSMENT TOOLS

1. Design and implement a **C-based validation module** that simulates a **Deterministic Finite Automaton (DFA)** to verify input strings in a real-time system. The module should accept only those strings that **begin with the character 'a' and end with the character 'a'**, ensuring compliance with predefined input format rules (e.g., protocol validation, pattern filtering, or secure data transmission).

The system should:

- Process user input dynamically
- Validate strings based on DFA state transitions
- Output whether the given input is **Accepted** or **Rejected**

2. Design a Push Down Automata that accepts the language

$$L = \{w \mid w \in (0 + 1)^* \text{ and } n_0(w) = n_1(w)\}$$

$n_0(w)$ is the number of 0's in w

$n_1(w)$ is the number of 1's in w

3. Design a Pushdown Automata for the language representing a's followed by b's and the number of b's must be twice that of a's.

$$L = \{a^n b^{2n} \mid n \geq 1\}$$

4. Develop a **C-based computational module** to determine the **ϵ -closure of all states** in a **Non-Deterministic Finite Automaton (NFA) with ϵ -transitions**, as part of an automata processing engine used in compiler construction and pattern-matching systems.

5. Develop a **C-based input validation module** that determines whether a given binary string conforms to a specified **Context-Free Grammar (CFG)** used in structured data validation systems.

The grammar is defined as:

- $S \rightarrow 0 A 1$
- $A \rightarrow 0 A \mid 1 A \mid \epsilon$

DFA - String starts and ends with a DFA

States:

q_0 = Start

q_1 = Accept

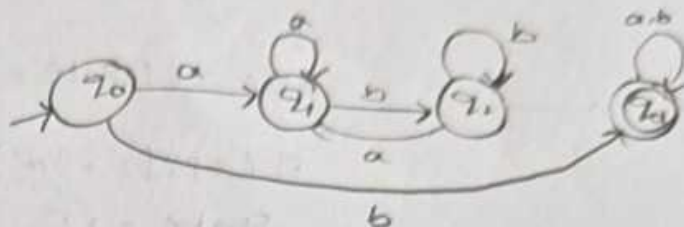
q_2 = Ends with b

q_d = Dead state

Transition Table

State	a	b
q_0	q_1	q_d
q_1	q_1	q_2
q_2	q_1	q_d
q_d	q_d	q_d

Diagram:



Example

Input: abba

Output: Accepted

Input: abb

Output: Rejected

Sample C program

```
#include <stdio.h>
#include <string.h>
int main()
{
    char s[100];
    int state = 0;
    printf("Enter string: ");
    scanf("%s", s);
    for (int i = 0; s[i] != '\0'; i++)
```



```

if (state == 0)
{
    if (s[i] == 'a')
        state = 1;
    else
        state = 3;
}
else if (state == 1)
{
    if (s[i] == 'a')
        state = 1;
    else
        state = 2;
}
else if (state == 2)
{
    if (s[i] == 'a')
        state = 1;
    else
        state = 2;
}
}
if (state == 1)
    printf("Accepted");
else
    printf("Rejected");
return 0;
}

```

2) PDA for Number of 0s and 1s

The PDF text for Q₂ is incomplete: it does not show the actual mathematical language.

So, do not write a guessed answer if your question

Paper has a missing formula.

If the intended language is:

$$L = \{0^n 1^n \mid n \geq 1\}$$

then use this simple PDA.

Idea

- for every 0, Push x
- for every 1, Pop x
- If all x's are removed, accept.

Transition Table

Current state	Input	State Top	Next state	Stack operation
q_0	0	z_0	q_0	xz_0
q_0	0	x	q_0	xx
q_0	1	x	q_1	e
q_0	1	z_0	q_1	e
q_1	1	x	q_1	e
q_1	e	z_0	q_f	z_0

Example

Input: 0011

Stack processing:

Read 0 → Push x

Read 0 → Push x

Read 1 → Pop x

Read 1 → Pop x

The input is completely consumed and the PDA reached the final state.

∴ 0011 → Accepted

But

00111 → Rejected

because there are more 1s than 0s.

③ PDA for $L = \{a^n b^{2n} \mid n \geq 1\}$

This question is explicitly given in your PDF.

Simple Idea

for every a:

Push 2x

for every b:

Pop 1x

The xxx notation means:

Existing top x

Read a

Push two additional xs

Example:

abb $\rightarrow$ Accepted

aabb $\rightarrow$ Accepted

aaabbbbbb $\rightarrow$ Accepted

ab $\rightarrow$ Rejected

aabb $\rightarrow$ Rejected

Transition

$$\delta(q_0, a, z_0) = (q_0, xxz_0)$$

$$\delta(q_0, a, x) = (q_0, xxx)$$

$$\delta(q_0, b, x) = (q_1, \epsilon)$$

$$\delta(q_1, b, x) = (q_1, \epsilon)$$

$$\delta(q_1, \epsilon, z_0) = (q_f, z_0)$$

4) ϵ -closure of NFA

The question asks to find the ϵ -closure of all states.

Definition

ϵ -closure means:

All states reachable from a state using only

ϵ -transitions.

Example

$$q_0 \xrightarrow{\epsilon} q_1 \xrightarrow{\epsilon} q_2$$

$$\therefore \epsilon\text{-closure}(q_0) = \{q_0, q_1, q_2\}$$

$$\epsilon\text{-closure}(q_1) = \{q_1, q_2\}$$

$$\epsilon\text{-closure}(q_2) = \{q_2\}$$

C program

```
#include <stdio.h>
```

```
int n;
```

```
int a[10][10];
```

```
int visited[10];
```

```
void closure(int state)
```

```
{
```

```
void closure(int state)
```

```
{
```

```
visited[state] = 1;
```

```
for (int i = 0; i < n; i++)
```

```
{
```

```
if (a[state][i] == 1 && visited[i] == 0)
```

```
{
```

```
closure(i);
```

```
}
```

```

int main()
{
    printf("Enter number of states: ");
    scanf("%d", &n);
    printf("Enter epsilon + transition matrix: \n");
    for (int i=0; i<n; i++)
        for (int j=0; j<n; j++)
            scanf("%d", &a[i][j]);

    for (int i=0; i<n; i++)
    {
        for (int j=0; j<n; j++)
        {
            visited[j]=0;
            closure(i);
            printf("E-closure(%d) = {", i);
            for (int j=0; j<n; j++)
            {
                if (visited[j])
                    printf("%d ", j);
            }
            printf("\n");
        }
        return 0;
    }
}

```

5) CFG Validation

Given grammar:

$S \rightarrow 0A1$

$A \rightarrow 0A1 \mid \epsilon$

Simple Explanation

The string must:

→ Start with 0

→ End with 1

Middle can contain 0 or 1

$\therefore$ 0 + any binary string + 1

Accepted examples

01

001

011

0101

00101

Rejected examples

0

1

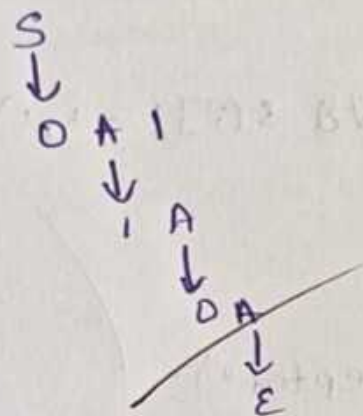
10

11

101

Simple parse

for: 0101



$\therefore$

$$\begin{aligned} S &\Rightarrow 0A1 \\ &\Rightarrow 01A1 \\ &\Rightarrow 010A1 \\ &\Rightarrow 0101 \end{aligned}$$

C programs

```
#include <stdio.h>
```

```
#include <string.h>
```

```
int main()
```

```
{
```

```
    char s[100];
```

```
    int n, valid = 1;
```

```
    printf("Enter binary string: ");
```

```
    scanf("%s", s);
```

```
    n = strlen(s);
```

```
    if (n < 2)
```

```
        valid = 0;
```

```
    if (s[0] != '0' || s[n-1] != '1')
```

```
        valid = 0;
```

```
    for (int i = 0; i < n; i++)
```

```
    {
```

```
        if (s[i] != '0' && s[i] != '1')
```

```
            valid = 0;
```

```
    }
```

```
    if (valid)
```

```
        printf("Accepted");
```

```
    else
```

```
        printf("Rejected");
```

```
    return 0;
```

```
}
```

SIMATS ENGINEERING ASSESSMENT TOOLS

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	25	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	20	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	15	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	10	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

92/100